

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 10
з курсу “Операційні системи реального часу”
на тему: “Розробка багатозадачної інтерактивної гри для міні-консолі”

Виконав:
Студент. гр. ФеП-41 Фіняк Олег
Перевірив:
Павлик М.Р.

Львів – 2025

Мета роботи. Навчитися розробляти прості інтерактивні ігри типу “пінг-понг” та “змійка” для міні-консолі з використанням механізмів RTOS.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "pico/stdlib.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"

/* OLED / GUI library headers from lab (as used in PDF) */
#include "DEV_Config.h"
#include "GUI_Paint.h"
#include "ImageData.h"
#include "Debug.h"

/* --- Configuration --- */
/* GPIO pins — змініть під своє підключення */
#define BTN_A_PIN 15 // кнопка А (вперед / select)
#define BTN_B_PIN 17 // кнопка В (назад / back)
#define JOY_X_PIN 26 // (при використанні аналогового) not used here
#define JOY_Y_PIN 27 // not used here
/* Якщо джойстик — 4 цифрові кнопки: */
#define JOY_UP_PIN 16
#define JOY_DOWN_PIN 18
#define JOY_LEFT_PIN 19
#define JOY_RIGHT_PIN 20

/* Timing */
#define POLL_PERIOD_MS 50 // опитування кнопок (50 ms)
#define GAME_TICK_MS 40 // ігровий кадр (25 fps -> 40 ms)
#define DISPLAY_PERIOD_MS 40 // частота оновлення дисплея

/* Display size — під OLED 1.3" з PDF */
#define OLED_W OLED_1in3_C_WIDTH
#define OLED_H OLED_1in3_C_HEIGHT

/* --- Data structures --- */
typedef enum {
    EVT_NONE = 0,
    EVT_BTN_A_PRESS,
    EVT_BTN_B_PRESS,
    EVT_JOY_UP,
    EVT_JOY_DOWN,
    EVT_JOY_LEFT,
    EVT_JOY_RIGHT,
    EVT_QUIT,
} input_event_t;

/* Game selection */
typedef enum {
    GAME_NONE = 0,
    GAME_PONG,
    GAME_SNAKE,
} game_id_t;
```

```

/* --- FreeRTOS objects --- */
static QueueHandle_t xInputQueue;
static SemaphoreHandle_t xFrameSem;    // бінарний семафор для нового кадру
static SemaphoreHandle_t xDisplayMutex; // mutex для буферу відображення

/* Frame buffer pointer (shared) */
static UBYTE *g_pFrameBuf = NULL;
static UWORD g_frame_size = 0;

/* Current game state container */
typedef struct {
    game_id_t current_game;
    bool running;
    bool paused;
    // game specific state can go here...
} app_state_t;

static app_state_t g_app = { .current_game = GAME_NONE, .running = true, .paused = false };

/* --- Forward declarations --- */
static void vTaskInputPoll(void *pvParameters);
static void vTaskGameLogic(void *pvParameters);
static void vTaskDisplay(void *pvParameters);
static void init_hw(void);
static void draw_main_menu(void);
static void draw_pause_menu(void);
static void pong_init(void);
static void pong_update(input_event_t ev);
static void pong_draw(void);
static void snake_init(void);
static void snake_update(input_event_t ev);
static void snake_draw(void);

/* --- Simple helper: digital read wrapper for DEV library if available --- */
static inline int read_pin(int gpio) {
    /* If using DEV_Digital_Read from lab library:
       return DEV_Digital_Read(gpio);
       but in many setups we use pico's gpio_get() (ACTIVE LOW/ HIGH may vary).
    */
    return gpio_get(gpio);
}

/* ----- Implementation ----- */

static void init_hw(void) {
    stdio_init_all();
    /* Initialize DEV module (OLED) from the lab. */
    DEV_Delay_ms(100);
    printf("Initializing display...\n");
    if (DEV_Module_Init() != 0) {
        printf("DEV_Module_Init failed!\n");
        // Hang
        while (1) tight_loop_contents();
    }
    OLED_1in3_C_Init();
    OLED_1in3_C_Clear();

    g_frame_size = ((OLED_W % 8 == 0) ? (OLED_W / 8) : (OLED_W / 8 + 1)) * OLED_H;
    g_pFrameBuf = (UBYTE *)malloc(g_frame_size);

```

```

if (!g_pFrameBuf) {
    printf("Failed to allocate framebuffer\n");
    while (1) tight_loop_contents();
}
Paint_NewImage(g_pFrameBuf, OLED_W, OLED_H, 0, WHITE);
Paint_Clear(BLACK);
OLED_1in3_C_Display(g_pFrameBuf);
printf("Display ready\n");

/* Configure GPIOs */
// Buttons and joystick pins - set as inputs with pull-ups
const int pins[] = {BTN_A_PIN, BTN_B_PIN, JOY_UP_PIN, JOY_DOWN_PIN, JOY_LEFT_PIN,
JOY_RIGHT_PIN};
for (size_t i = 0; i < sizeof(pins)/sizeof(pins[0]); ++i) {
    gpio_init(pins[i]);
    gpio_set_dir(pins[i], GPIO_IN);
    gpio_pull_up(pins[i]); // assuming active-low buttons (press -> 0). If opposite, change logic.
}
}

/* Debounce helper: simple stateful debounce per pin */
typedef struct {
    uint32_t pin;
    bool stable_state; // last stable logical level (pressed/not pressed)
    uint32_t counter; // ms counter
    uint32_t last_sample_time;
} debounce_t;

static void vTaskInputPoll(void *pvParameters) {
    const TickType_t xPeriod = pdMS_TO_TICKS(POLL_PERIOD_MS);
    TickType_t xLastWake = xTaskGetTickCount();

    // For simplicity use 6 debouncers
    debounce_t db[6] = {
        { .pin = BTN_A_PIN, .stable_state = 1, .counter = 0 },
        { .pin = BTN_B_PIN, .stable_state = 1, .counter = 0 },
        { .pin = JOY_UP_PIN, .stable_state = 1, .counter = 0 },
        { .pin = JOY_DOWN_PIN, .stable_state = 1, .counter = 0 },
        { .pin = JOY_LEFT_PIN, .stable_state = 1, .counter = 0 },
        { .pin = JOY_RIGHT_PIN, .stable_state = 1, .counter = 0 },
    };

    const uint32_t debounce_ms = 50; // require stable for 50ms

    while (1) {
        vTaskDelayUntil(&xLastWake, xPeriod);

        for (int i = 0; i < 6; ++i) {
            int raw = gpio_get(db[i].pin); // 0 when pressed if pull-up used
            // Normalize to logical pressed (true if pressed)
            bool is_pressed = (raw == 0);

            // If changed from stable, increment counter, else reset to 0
            if ((int)is_pressed != (int)db[i].stable_state) {
                db[i].counter += POLL_PERIOD_MS;
                if (db[i].counter >= debounce_ms) {
                    // State flipped
                    db[i].stable_state = is_pressed;
                    db[i].counter = 0;
                }
            }
        }
    }
}

```

```

// Create event when pressed (not released)
if (is_pressed) {
    input_event_t ev = EVT_NONE;
    switch (i) {
        case 0: ev = EVT_BTN_A_PRESS; break;
        case 1: ev = EVT_BTN_B_PRESS; break;
        case 2: ev = EVT_JOY_UP; break;
        case 3: ev = EVT_JOY_DOWN; break;
        case 4: ev = EVT_JOY_LEFT; break;
        case 5: ev = EVT_JOY_RIGHT; break;
    }
    if (ev != EVT_NONE) {
        // send event (non-blocking)
        xQueueSendToBack(xInputQueue, &ev, 0);
    }
}
} else {
    db[i].counter = 0;
}
}
}
}

```

/* --- Simple Pong implementation --- */

```

static int pong_ball_x, pong_ball_y, pong_dir_x, pong_dir_y;
static int pong_player_y, pong_ai_y;
static int pong_player_score, pong_ai_score;
static const int PADDLE_H = 18;
static const int PADDLE_W = 3;

```

```

static void pong_init(void) {
    pong_ball_x = OLED_W/2;
    pong_ball_y = OLED_H/2;
    pong_dir_x = 1; pong_dir_y = 1;
    pong_player_y = OLED_H/2 - PADDLE_H/2;
    pong_ai_y = pong_player_y;
    pong_player_score = 0; pong_ai_score = 0;
}

```

```

static void pong_update(input_event_t ev) {
    // Move player paddle
    if (ev == EVT_JOY_UP) {
        pong_player_y -= 6;
        if (pong_player_y < 0) pong_player_y = 0;
    } else if (ev == EVT_JOY_DOWN) {
        pong_player_y += 6;
        if (pong_player_y > OLED_H - PADDLE_H) pong_player_y = OLED_H - PADDLE_H;
    }
    // Update ball position
    pong_ball_x += pong_dir_x * 2;
    pong_ball_y += pong_dir_y * 2;
    // Collisions top/bottom
    if (pong_ball_y <= 0 || pong_ball_y >= OLED_H-1) pong_dir_y = -pong_dir_y;
    // Player paddle collision (left side)
    if (pong_ball_x <= (10 + PADDLE_W) && pong_ball_x >= 10) {
        if (pong_ball_y >= pong_player_y && pong_ball_y <= pong_player_y + PADDLE_H) {
            pong_dir_x = 1;

```

```

    }
}
// AI paddle moves simply towards ball
if (pong_ai_y + PADDLE_H/2 < pong_ball_y) pong_ai_y += 2;
else pong_ai_y -= 2;
if (pong_ai_y < 0) pong_ai_y = 0;
if (pong_ai_y > OLED_H - PADDLE_H) pong_ai_y = OLED_H - PADDLE_H;
// AI paddle collision (right side)
if (pong_ball_x >= OLED_W - (10 + PADDLE_W) && pong_ball_x <= OLED_W - 10) {
    if (pong_ball_y >= pong_ai_y && pong_ball_y <= pong_ai_y + PADDLE_H) {
        pong_dir_x = -1;
    }
}
}
// Scoring
if (pong_ball_x < 0) {
    pong_ai_score++;
    pong_init();
} else if (pong_ball_x > OLED_W) {
    pong_player_score++;
    pong_init();
}
}

static void pong_draw(void) {
    Paint_Clear(BLACK);
    // Middle line
    for (int y = 0; y < OLED_H; y += 4) {
        Paint_DrawPoint(OLED_W/2, y, WHITE, DOT_PIXEL_1X1, DOT_STYLE_DFT);
    }
    // Player paddle (left)
    Paint_DrawRectangle(10, pong_player_y, 10 + PADDLE_W, pong_player_y + PADDLE_H, WHITE,
DOT_PIXEL_1X1, DRAW_FILL_FULL);
    // AI paddle (right)
    Paint_DrawRectangle(OLED_W - 10 - PADDLE_W, pong_ai_y, OLED_W - 10, pong_ai_y +
PADDLE_H, WHITE, DOT_PIXEL_1X1, DRAW_FILL_FULL);
    // Ball
    Paint_DrawCircle(pong_ball_x, pong_ball_y, 2, WHITE, DOT_PIXEL_1X1, DRAW_FILL_FULL);
    // Scores
    char s[16];
    sprintf(s, "%d", pong_player_score);
    Paint_DrawString_EN(20, 2, s, &Font12, WHITE, BLACK);
    sprintf(s, "%d", pong_ai_score);
    Paint_DrawString_EN(OLED_W - 30, 2, s, &Font12, WHITE, BLACK);
}

/* --- Simple Snake implementation --- */
#define SNAKE_MAX 200
static int snake_len;
static int snake_x[SNAKE_MAX], snake_y[SNAKE_MAX];
static int snake_dx, snake_dy;
static int snake_food_x, snake_food_y;
static bool snake_alive;

static void snake_place_food(void) {
    snake_food_x = 5 + rand() % (OLED_W - 10);
    snake_food_y = 5 + rand() % (OLED_H - 10);
}

static void snake_init(void) {

```

```

snake_len = 5;
int startx = OLED_W/2, starty = OLED_H/2;
for (int i = 0; i < snake_len; ++i) {
    snake_x[i] = startx - i*4;
    snake_y[i] = starty;
}
snake_dx = 4; snake_dy = 0;
snake_place_food();
snake_alive = true;
}

static void snake_update(input_event_t ev) {
    if (!snake_alive) return;
    if (ev == EVT_JOY_LEFT && !(snake_dx > 0)) { snake_dx = -4; snake_dy = 0; }
    if (ev == EVT_JOY_RIGHT && !(snake_dx < 0)) { snake_dx = 4; snake_dy = 0; }
    if (ev == EVT_JOY_UP && !(snake_dy > 0)) { snake_dx = 0; snake_dy = -4; }
    if (ev == EVT_JOY_DOWN && !(snake_dy < 0)) { snake_dx = 0; snake_dy = 4; }

    // Move body
    for (int i = snake_len - 1; i > 0; --i) {
        snake_x[i] = snake_x[i-1];
        snake_y[i] = snake_y[i-1];
    }
    snake_x[0] += snake_dx;
    snake_y[0] += snake_dy;
    // Wall collision
    if (snake_x[0] < 0 || snake_x[0] >= OLED_W || snake_y[0] < 0 || snake_y[0] >= OLED_H) {
        snake_alive = false;
        return;
    }
    // Self collision
    for (int i = 1; i < snake_len; ++i) {
        if (abs(snake_x[0] - snake_x[i]) < 3 && abs(snake_y[0] - snake_y[i]) < 3) {
            snake_alive = false;
            return;
        }
    }
    // Food
    if (abs(snake_x[0] - snake_food_x) < 4 && abs(snake_y[0] - snake_food_y) < 4) {
        if (snake_len < SNAKE_MAX-1) {
            snake_x[snake_len] = snake_x[snake_len-1];
            snake_y[snake_len] = snake_y[snake_len-1];
            snake_len++;
            snake_place_food();
        }
    }
}

static void snake_draw(void) {
    Paint_Clear(BLACK);
    // Draw food
    Paint_DrawRectangle(snake_food_x-2, snake_food_y-2, snake_food_x+2, snake_food_y+2, WHITE,
DOT_PIXEL_1X1, DRAW_FILL_FULL);
    // Draw snake
    for (int i = 0; i < snake_len; ++i) {
        Paint_DrawRectangle(snake_x[i]-2, snake_y[i]-2, snake_x[i]+2, snake_y[i]+2, WHITE,
DOT_PIXEL_1X1, DRAW_FILL_FULL);
    }
    if (!snake_alive) {

```

```

        Paint_DrawString_EN(10, 5, "Game Over", &Font12, WHITE, BLACK);
    }
}

/* --- Menu drawing --- */
static void draw_main_menu(void) {
    Paint_Clear(BLACK);
    Paint_DrawString_EN(10, 10, "Mini Console", &Font16, WHITE, BLACK);
    Paint_DrawString_EN(10, 32, "A: Select B: Back", &Font8, WHITE, BLACK);
    Paint_DrawString_EN(10, 46, "Up/Down: Menu", &Font8, WHITE, BLACK);
    Paint_DrawString_EN(10, 70, "1. Pong", &Font12, WHITE, BLACK);
    Paint_DrawString_EN(10, 85, "2. Snake", &Font12, WHITE, BLACK);
}

/* Pause menu */
static void draw_pause_menu(void) {
    Paint_Clear(BLACK);
    Paint_DrawString_EN(10, 10, "Paused", &Font24, WHITE, BLACK);
    Paint_DrawString_EN(10, 40, "A: Resume B: Quit", &Font12, WHITE, BLACK);
}

/* --- Game logic task --- */
static void vTaskGameLogic(void *pvParameters) {
    TickType_t xLastWake = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(GAME_TICK_MS);

    int menu_sel = 0; // 0->Pong, 1->Snake
    draw_main_menu();
    // Display initial menu
    xSemaphoreTake(xDisplayMutex, portMAX_DELAY);
    OLED_1in3_C_Display(g_pFrameBuf);
    xSemaphoreGive(xDisplayMutex);

    while (1) {
        vTaskDelayUntil(&xLastWake, xPeriod);

        // Process all pending input events
        input_event_t ev;
        while (xQueueReceive(xInputQueue, &ev, 0) == pdTRUE) {
            // Global controls
            if (g_app.current_game == GAME_NONE) {
                // Menu navigation
                if (ev == EVT_JOY_UP) { if (menu_sel > 0) menu_sel--; }
                if (ev == EVT_JOY_DOWN) { if (menu_sel < 1) menu_sel++; }
                if (ev == EVT_BTN_A_PRESS) {
                    // select
                    if (menu_sel == 0) {
                        g_app.current_game = GAME_PONG;
                        pong_init();
                    } else {
                        g_app.current_game = GAME_SNAKE;
                        snake_init();
                    }
                    g_app.paused = false;
                }
                if (ev == EVT_BTN_B_PRESS) {
                    // nothing to go back to — could exit app
                }
            } else {

```



```

// In-game controls
if (ev == EVT_BTN_B_PRESS) {
    // pause menu
    g_app.paused = !g_app.paused;
} else if (ev == EVT_BTN_A_PRESS) {
    // could be used for actions
} else {
    // pass joystick events to game
    if (!g_app.paused) {
        if (g_app.current_game == GAME_PONG) {
            pong_update(ev);
        } else if (g_app.current_game == GAME_SNAKE) {
            snake_update(ev);
        }
    } else {
        // paused: A resume, B quit
        if (ev == EVT_BTN_A_PRESS) {
            g_app.paused = false;
        } else if (ev == EVT_BTN_B_PRESS) {
            // quit to menu
            g_app.current_game = GAME_NONE;
            draw_main_menu();
            xSemaphoreTake(xDisplayMutex, portMAX_DELAY);
            OLED_1in3_C_Display(g_pFrameBuf);
            xSemaphoreGive(xDisplayMutex);
        }
    }
}
} // end processing events

// If no game selected - draw menu highlight
if (g_app.current_game == GAME_NONE) {
    // small visual menu highlight
    Paint_Clear(BLACK);
    Paint_DrawString_EN(10, 10, "Mini Console", &Font16, WHITE, BLACK);
    Paint_DrawString_EN(10, 32, "A: Select B: Back", &Font8, WHITE, BLACK);
    Paint_DrawString_EN(10, 46, "Up/Down: Menu", &Font8, WHITE, BLACK);
    if (menu_sel == 0) {
        Paint_DrawString_EN(10, 70, "> Pong", &Font12, WHITE, BLACK);
        Paint_DrawString_EN(10, 85, " Snake", &Font12, WHITE, BLACK);
    } else {
        Paint_DrawString_EN(10, 70, " Pong", &Font12, WHITE, BLACK);
        Paint_DrawString_EN(10, 85, "> Snake", &Font12, WHITE, BLACK);
    }
    // give display semaphore
    xSemaphoreGive(xFrameSem);
    continue;
}

// If paused, show pause menu and signal frame
if (g_app.paused) {
    draw_pause_menu();
    xSemaphoreGive(xFrameSem);
    continue;
}

// Game update step
if (g_app.current_game == GAME_PONG) {

```

```

        // pong_update() was called on joystick events; still update physics each tick
        pong_update(EVT_NONE);
        // Draw into framebuffer
        pong_draw();
    } else if (g_app.current_game == GAME_SNAKE) {
        snake_update(EVT_NONE);
        snake_draw();
    }
    // Signal display task that a new frame is ready
    xSemaphoreGive(xFrameSem);
}
}

/* --- Display task --- */
static void vTaskDisplay(void *pvParameters) {
    while (1) {
        // Wait until a frame is ready
        if (xSemaphoreTake(xFrameSem, portMAX_DELAY) == pdTRUE) {
            // Lock framebuffer for display
            if (xSemaphoreTake(xDisplayMutex, pdMS_TO_TICKS(1000)) == pdTRUE) {
                OLED_1in3_C_Display(g_pFrameBuf);
                xSemaphoreGive(xDisplayMutex);
            } else {
                // couldn't get mutex; skip frame
            }
            // small delay to avoid busy loop
            vTaskDelay(pdMS_TO_TICKS(DISPLAY_PERIOD_MS/2));
        }
    }
}

/* --- Main --- */
int main() {
    init_hw();

    /* Create synchronization objects */
    xInputQueue = xQueueCreate(16, sizeof(input_event_t));
    xFrameSem = xSemaphoreCreateBinary();
    xDisplayMutex = xSemaphoreCreateMutex();

    if (!xInputQueue || !xFrameSem || !xDisplayMutex) {
        printf("Failed to create RTOS objects\n");
        while (1) tight_loop_contents();
    }

    /* Start tasks */
    BaseType_t r;
    r = xTaskCreate(vTaskInputPoll, "InputPoll", 512, NULL, 3, NULL);
    r |= xTaskCreate(vTaskGameLogic, "GameLogic", 2048, NULL, 2, NULL);
    r |= xTaskCreate(vTaskDisplay, "Display", 1024, NULL, 1, NULL);
    if (r != pdPASS) {
        printf("Task create failed\n");
        while (1) tight_loop_contents();
    }

    /* Initially put main menu frame */
    draw_main_menu();
    xSemaphoreGive(xFrameSem);
}

```

```
/* Start the scheduler */  
vTaskStartScheduler();  
  
/* Should never reach here */  
for (;;) ;  
return 0;  
}
```

Висновок: У ході лабораторної роботи було створено багатозадачну міні-консоль на FreeRTOS, де кожна частина системи жила у своєму власному ритмі: опитування кнопок, ігрова логіка та рендер кадрів. Вдалося реалізувати черги, семафори і м'ютекси, перетворивши хаос апаратних подій на кероване і пружне ігрове середовище. До консолі було додано меню, паузу та кілька повноцінних ігор, що реагують на користувача в реальному часі. Робота показала, як RTOS дозволяє створювати плавні, швидкі і чітко синхронізовані інтерактивні застосунки на мікроконтролерах.